

به نام خدا



هوش مصنوعی
نیم سال دوم ۰۴-۰۵
استاد: دکتر نصیری مفخم
شماره گروه : ۱۸
اعضای گروه:

فرنوش پورشبان (۴۰۲۴۰۲۳۰۰۷)
فرزانه سلیمی (۴۰۱۴۰۱۳۰۵۹)
یگانه رستگاری (۴۰۱۴۰۱۳۰۴۰)

دانشکده ریاضی و آمار

پروژه سری دوم

تعریف پروژه:

در پروژه آخرین سیگنال، با یک مسئله بهینه‌سازی در محیط شبکه شطرنجی (Grid World) مواجه هستیم که هدف آن، جانمایی بهینه تعداد محدودی سنسور برای پوشش حداکثری اهداف (Targets) و اجتناب از موانع (Obstacles) است. در الگوریتم‌های جستجوی محلی برای مسئله مسیر رسیدن به جواب اهمیتی ندارد، بلکه خود وضعیت نهایی (چیدمان سنسورها) هدف اصلی است. در ادامه با چند الگوریتم جستجوی محلی جواب‌ها را بررسی می‌کنیم.

لینک گیت‌هاب پروژه:

<https://github.com/Farnoush2005/AI-Sensor-Placement-Project>

الگوریتم Local Search Base :

کارکرد الگوریتم:

این کلاس به عنوان یک کلاس والد (Base Class) طراحی شده تا مفاهیم و توابع کلیدی که بین همه الگوریتم‌های جستجوی محلی مشترک هستند را هماهنگ کند. این الگوریتم دو کار اساسی انجام می‌دهد:

۱. با استفاده از تابع neighbor function همسایه تولید می‌کند.

۲. پس از مقداردهی اولیه و جایگذاری تصادفی سنسورها، با استفاده از تابع هزینه (cost) که در آن تعریف می‌کنیم چینش مناسب را تعیین می‌کند.

تعریف‌های اولیه :

در این بخش ابتدا باید توابع ریاضی و معیاری را برای سنجش تعریف کنیم.

۱. تابع هزینه:

$$\text{cost} = (\text{تعداد سنسورهای استفاده شده در حالت فعلی} * 10) + (\text{تعداد هدف های پوشش داده نشده} * 100)$$

وزن سنگین جریمه‌ی اهداف (۱۰۰ در مقابل ۱۰) به این دلیل است که می‌خواهیم اولویت اول سیستم، تحت پوشش قرار دادن تمام اهداف باشد و اولویت دوم، بهینه‌سازی و کم کردن تعداد سنسورها.

۲. برای بررسی اینکه آیا یک هدف تحت پوشش سنسور هست یا خیر، معیاری تعریف می‌کنیم و برای این کار از فاصله‌ی منهتن (Manhattan Distance) استفاده می‌کنیم:

$$\text{Distance} = |x_{\text{target}} - x_{\text{sensor}}| + |y_{\text{target}} - y_{\text{sensor}}|$$

اگر فاصله (Distance) از برد سنسور (sensor_range) کمتر یا مساوی باشد، این یعنی هدف پوشش داده شده است.

توضیح و تحلیل کد الگوریتم:

ابتدا کلاس Local Search Base را تعریف کرده و شیء‌های آن را به صورت زیر تعریف می‌کنیم:

- `self.max_sensors`: حداکثر تعداد مجاز سنسورها در نقشه.
- `self.sensor_range`: شعاع برد سنسورها.
- `self.targets`: مختصات تمام اهدافی که باید پوشش داده شوند.

سپس آن‌ها را مقداردهی می‌کنیم.

```
class LocalSearchBase:
    def __init__(self, world):
        self.world = world
        self.max_sensors = world.max_sensors
        self.sensor_range = world.sensor_range
        self.targets = self.world.get_targets()
```

تابع `evaluate` را تعریف کرده و در آن معیارهای سنجش (که در قسمت قبل توضیح داده شد) را مشخص می‌کنیم:

```
def evaluate(self, state):
    covered = set()

    # بررسی پوشش تارگتها با فاصله منتهن
    for t in self.targets:
        for s in state:
            dist = abs(t[0] - s[0]) + abs(t[1] - s[1])
            if dist <= self.sensor_range:
                covered.add(t)
                break

    # فرمول هزینه: هر تارگت جریمه ۱۰۰ و هر سنسور جریمه ۱۰
    uncovered_count = len(self.targets) - len(covered)
    return (uncovered_count * 100) + (len(state) * 10)
```

با استفاده از تابع همسایگی (`get_neighbor`) همسایه تولید می‌کنیم. این تابع نحوه‌ی حرکت در فضای حالت از طریق تغییرات کوچک را مشخص می‌کند و با ۳ عملگر (`move`, `add`, `remove`) مجاز همسایه‌های یک حالت را تولید می‌کند:

```
def get_neighbor(self, state):
    # مشخص کردن کارهای مجاز بر اساس محدودیتها
    ops = ['move']
    if len(state) < self.max_sensors:
        ops.append('add')
    if len(state) > 1:
        ops.append('remove')

    op = random.choice(ops)
    new_state = copy.deepcopy(state)
```

move: یکی از سنسورها را به صورت تصادفی انتخاب کرده و به یکی از ۴ جهت اصلی (بالا، پایین، چپ، راست) جابجا می‌کند. اگر اطراف سنسور بسته بود، یک موقعیت خالی تصادفی در کل نقشه برای آن پیدا می‌کند.

```
if op == 'move':
    idx = random.randint(0, len(new_state) - 1)
    cx, cy = new_state[idx]

    # جابجایی در ۴ جهت اصلی به صورت تصادفی
    dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    random.shuffle(dirs)
    moved = False

    for dx, dy in dirs:
        nx, ny = cx + dx, cy + dy
        if self.world.is_valid_position(nx, ny) and (nx, ny) not in new_state:
            new_state[idx] = (nx, ny)
            moved = True
            break

    # اگر دورش بسته بود، به جای تصادفی توی مپ براش پیدا میکنیم
    if not moved:
        for _ in range(50):
            nx = random.randint(0, self.world.rows - 1)
            ny = random.randint(0, self.world.cols - 1)
            if self.world.is_valid_position(nx, ny) and (nx, ny) not in new_state:
                new_state[idx] = (nx, ny)
                break
```

add: اگر هنوز به سقف مجاز تعداد سنسور نرسیده باشیم، یک سنسور جدید در یک مختصات خالی و معتبر ایجاد می‌کند.

```
elif op == 'add':
    # اضافه کردن سنسور جدید در یک موقعیت خالی و معتبر
    for _ in range(100):
        nx = random.randint(0, self.world.rows - 1)
        ny = random.randint(0, self.world.cols - 1)
        if self.world.is_valid_position(nx, ny) and (nx, ny) not in new_state:
            new_state.append((nx, ny))
            break
```

remove: اگر بیش از یک سنسور داشته باشیم، یکی را به تصادف حذف می‌کند تا هزینه سنسورها کاهش یابد.

```
elif op == 'remove':
    # حذف تصادفی یکی از سنسورها
    idx = random.randint(0, len(new_state) - 1)
    new_state.pop(idx)

    return new_state
```

الگوریتم‌های جستجوی محلی برای شروع نیاز به یک نقطه اولیه در فضای حالت دارند. پس باید حالت اولیه تصادفی را با تابع `initialize_state` برای شروع ایجاد کنیم که به شکل زیر عمل می‌کند.

ابتدا تعداد سنسورهای اولیه را به صورت تصادفی بین 1 تا `max_sensors` تعیین می‌کند. با استفاده از یک حلقه‌ی `while` کنترل‌شده (حداکثر ۱۰۰۰ تلاش برای جلوگیری از حلقه‌ی بی‌نهایت در نقشه‌های پر)، مختصات‌های تصادفی و معتبری که مانع نیستند (`is_valid_position`) و قبلاً سنسوری در آن‌ها قرار نگرفته باشد را انتخاب کرده و چیدمان اولیه را می‌سازد.

```
def initialize_state(self):
    # ساخت چیدمان اولیه تصادفی
    state = []
    num_sensors = random.randint(1, self.max_sensors)
    attempts = 0

    while len(state) < num_sensors and attempts < 1000:
        nx = random.randint(0, self.world.rows - 1)
        ny = random.randint(0, self.world.cols - 1)
        if self.world.is_valid_position(nx, ny) and (nx, ny) not in state:
            state.append((nx, ny))
            attempts += 1

    return state
```

الگوریتم Hill Climbing:

کارکرد الگوریتم:

در الگوریتم Hill Climbing با شروع از یک نقطه (حالت اولیه) و حرکت مداوم به سمت همسایه‌هایی با وضعیت بهتر، تلاش می‌کند به پاسخ بهینه برسد. از آنجا که در این پروژه سنجش بهینگی جواب بر اساس تابع هزینه (Cost) است، حرکت الگوریتم به سمت همسایه‌هایی با هزینه‌ی کمتر خواهد بود.

توضیح و تحلیل کد الگوریتم:

ابتدا کلاس Hill Climbing را تعریف می‌کنیم و با استفاده از متدهای کلاس `LocalSearchBase` عملیات را انجام می‌دهیم. در واقع Hill Climbing متدهای `LocalSearchBase` را به ارث می‌برد.

در الگوریتم Hill Climbing، تمام همسایه‌های یک حالت تولید و ارزیابی می‌شوند. اما در پیاده‌سازی، از استراتژی نمونه‌برداری تصادفی محلی استفاده می‌شود. این کار به کمک دو پارامتر اصلی در تابع run انجام می‌شود:

1. max_iterations: سقف تعداد گام‌های مجاز برای حرکت الگوریتم (جلوگیری از حلقه‌های بی‌نهایت).
2. sample_size: تعداد همسایه‌هایی که در هر گام به صورت تصادفی تولید و بررسی می‌شوند (در اینجا به طور پیش‌فرض ۳۰ همسایه در نظر گرفتیم).

در شروع (همان طور که در کد زیر داریم)، حالت اولیه کپی شده و هزینه آن محاسبه می‌شود. دو لیست evaluations (برای ذخیره روند تغییرات هزینه) و states_history (برای ذخیره گام به گام تغییرات چیدمان سنسورها جهت تحلیل‌های بعدی) ایجاد می‌شوند.

```
import copy
from search.local_search_base import LocalSearchBase

class HillClimbing(LocalSearchBase):
    def run(self, initial_state, max_iterations=1000, sample_size=30, **kwargs):
        curr_state = copy.deepcopy(initial_state)
        curr_cost = self.evaluate(curr_state)

        evaluations = [curr_cost]
        states_history = [copy.deepcopy(curr_state)]
```

در ادامه الگوریتم در هر گام، به تعداد sample_size (مثلاً ۳۰ بار) متد get_neighbor را صدا می‌زند. این متد بر اساس کلاس LocalSearchBase، یکی از کارهای جابه‌جایی، حذف یا اضافه کردن سنسور را انجام می‌دهد. در پایان، این حلقه بهترین همسایه (best_nb) و کمترین هزینه‌ی پیدا شده در این نمونه‌برداری مشخص می‌کند.

```
for _ in range(max_iterations):
    best_nb = None
    best_nb_cost = float('inf')

    # نمونه‌برداری تصادفی از همسایه‌ها برای پیدا کردن بهترین‌شون
    for _ in range(sample_size):
        nb = self.get_neighbor(curr_state)
        nb_cost = self.evaluate(nb)

        if nb_cost < best_nb_cost:
            best_nb = nb
            best_nb_cost = nb_cost
```

سپس با شرط بررسی می‌کنیم اگر هزینه‌ی بهترین همسایه‌ی پیدا شده، بزرگتر یا مساوی هزینه‌ی حالت فعلی باشد، الگوریتم بلافاصله متوقف می‌شود (break).

```
# اگر همسایه‌ها بهتر نباشند، یعنی تو بهینه محلی گیر کردیم و تمام  
if best_nb_cost >= curr_cost:  
    break
```

در نهایت مقادیر را به روز می‌کنیم و الگوریتم یک تاپل با ۴ عنصر را بازمی‌گرداند:

- **curr_state**: چیدمان نهایی و بهینه‌ی سنسورها.
- **curr_cost**: هزینه‌ی نهایی چیدمان.
- **evaluations**: آرایه‌ای از هزینه‌ها در طول زمان که برای رسم نمودار همگرایی استفاده می‌شود.
- **states_history**: تاریخچه‌ی موقعیت‌ها برای خطایابی و تصویرسازی مپ.

```
curr_state = best_nb  
curr_cost = best_nb_cost  
  
evaluations.append(curr_cost)  
states_history.append(copy.deepcopy(curr_state))  
  
return curr_state, curr_cost, evaluations, states_history
```

نقاط قوت و ضعف الگوریتم:

نقاط قوت:

- سرعت بالا و صرفه‌جویی در محاسبات: با محدود کردن بررسی به ۳۰ همسایه (به جای بررسی تمام ترکیبات جابه‌جایی، حذف، اضافه در مپ)، الگوریتم در نقشه‌های بزرگ بسیار سریع عمل می‌کند.
- حافظه بهینه: فقط اطلاعات گام‌های پذیرفته شده و تاریخچه اصلی حفظ می‌شود.

نقاط ضعف:

- وابستگی به حالت اولیه (Initial State Dependency): اگر حالت اولیه تصادفی در موقعیت بدی از نقشه ایجاد شود، این الگوریتم به سرعت در یک نقطه‌ی بهینه محلی ضعیف (مثلاً با تعداد زیادی اهداف پوشش‌نداده) گیر می‌افتد و به جواب بهینه نمی‌رسد. این الگوریتم مکانیزمی برای پذیرش موقت پاسخ‌های بدتر (برای خارج شدن از نقطه‌ی بهینه محلی و در نهایت یافتن نقطه بهینه سراسری) ندارد.

الگوریتم Simulated Annealing :

کارکرد الگوریتم:

در الگوریتم Simulated Annealing (SA) سعی می‌کنیم که نقطه‌ی ضعیف الگوریتم Hill Climbing (وابستگی به حالت اولیه و عدم پذیرش موقت پاسخ بدتر) را درست کنیم. برای این کار ابتدا متغیرهای زیر را تعریف می‌کنیم:

- حالت سیستم (State): چینش سنسورها در مپ.
- انرژی سیستم (Energy): هزینه‌ی چینش (Cost). هدف ما رسیدن به حالت کمترین انرژی است.
- دما (Temperature): یک پارامتر کنترلی که در ابتدا بالا است و اجازه حرکت‌های تصادفی بزرگ را می‌دهد و به مرور زمان کاهش می‌یابد.

برای آنکه بتوانیم از بهینه محلی خارج شویم، SA حرکات رو به عقب (پذیرش پاسخ‌های بدتر) را تحت شرایط خاصی و با یک احتمال مشخص مجاز می‌داند. این کار باعث می‌شود الگوریتم بتواند از چاله‌های بهینه محلی خارج شود و احتمال بیشتری در یافتن بهینه سراسری (Global Minimum) داشته باشد.

این احتمال بر اساس قانون توزیع بولتزمن (Metropolis Acceptance Criterion) محاسبه می‌شود که به شکل زیر است:

$$P = \frac{1}{e^{\frac{\Delta Cost}{T}}}$$

توضیح و تحلیل کد الگوریتم:

ابتدا کلاس Simulated Annealing را تعریف می‌کنیم به صورتی که متدهای کلاس LocalSearchBase را به ارث می‌برد و متغیرها را تعریف می‌کنیم.

```

class SimulatedAnnealing(LocalSearchBase):
    def run(self, initial_state, initial_temp=1000.0, cooling_rate=0.99, min_temp=0.1, **kwargs):
        curr_state = copy.deepcopy(initial_state)
        curr_cost = self.evaluate(curr_state)

        best_state = copy.deepcopy(curr_state)
        best_cost = curr_cost

        evaluations = [curr_cost]
        states_history = [copy.deepcopy(curr_state)]

        t = initial_temp

```

پارامترهای ورودی تابع run به صورت زیر است:

- `initial_temp=1000.0`: دمای اولیه سیستم که پتانسیل کاوش را در ابتدا بالا نگه می‌دارد.
- `cooling_rate=0.99`: نرخ سرد شدن هندسی سیستم. در هر گام، دمای فعلی در این عدد ضرب می‌شود تا دما به آرامی کاهش یابد. در واقع با جلو رفتن الگوریتم، اثر بخش متغیر دما کم می‌شود.
- `min_temp=0.1`: دمای آستانه یا شرط توقف الگوریتم است.

در ادامه کد با نمونه برداری و محاسبه‌ی هزینه شروع می‌شود.

الگوریتم تا زمانی که دما به زیر `min_temp` نرسد به کار خود ادامه می‌دهد. در هر تکرار، یک همسایه تولید شده و اختلاف هزینه آن با حالت فعلی سنجیده می‌شود. اگر همسایه بهتر (هزینه کمتر) باشد آنگاه مقدار `delta` منفی خواهد بود و اگر همسایه بدتر (هزینه بیشتر) باشد آنگاه مقدار `delta` مثبت خواهد بود.

```

while t > min_temp:
    nb = self.get_neighbor(curr_state)
    nb_cost = self.evaluate(nb)

    delta = nb_cost - curr_cost

```

در این قسمت کد، تابع بولتزمن را تعریف کرده و مقایسه را انجام می‌دهیم. اگر $\delta > 0$ باشد یعنی همسایه جدید هزینه‌ی کمتری دارد (بهتر است)، پس پذیرفته می‌شود. اگر $\delta < 0$ باشد یعنی همسایه جدید بدتر است. در این حالت عبارت $\text{math.exp}(-\delta/t)$ محاسبه می‌شود که عددی بین ۰ و ۱ است. سپس با `random.random()` یک عدد تصادفی تولید می‌کنیم و مقایسه می‌کنیم اگر عدد تصادفی کوچکتر از احتمال بدست آمده باشد، الگوریتم پاسخ بدتر را می‌پذیرد تا از بهینه محلی خارج شود.

```
# اگر همسایه بهتر بود یا شانس آوردیم (بر اساس دما)، قبولش میکنیم
if delta < 0 or random.random() < math.exp(-delta / t):
    curr_state = nb
    curr_cost = nb_cost
```

در ابتدای اجرای الگوریتم که دما (T) بسیار بزرگ است، مخرج کسر بزرگ بوده و حاصل عبارت $e^{-\frac{\Delta}{T}}$ به عدد ۱ نزدیک تر است. بنابراین شانس پذیرش پاسخهای بدتر بالا است. اما به مرور زمان که سیستم سرد می شود و T کاهش می یابد، احتمال پذیرش پاسخهای بدتر به صفر نزدیک می شود و در اینجا الگوریتم مانند الگوریتم Hill climbing حریرانه روی بهره برداری از موقعیت های کشف شده تمرکز می کند.

از آنجا که این الگوریتم مسیر نوسانی دارد و ممکن است برای فرار از بهینه های محلی موقتاً به حالت های بد برود، یک متغیر جداگانه به نام **best_state** تعبیه می کنیم تا بهترین چیدمانی که سیستم در کل طول مسیر خود تجربه کرده را نگه دارد و در پایان برگرداند.

```
# آپدیت کردن بهترین جوابی که تا الان دیدیم
if curr_cost < best_cost:
    best_state = copy.deepcopy(curr_state)
    best_cost = curr_cost
```

در پایان دما را سرد میکنیم و مانند الگوریتم Hill Climbig، خروجی یک تاپل حاوی ۴ عنصر به صورت زیر است:

- **curr_state**: چیدمان نهایی و بهینه ی سنسورها.
- **curr_cost**: هزینه ی نهایی چیدمان.
- **evaluations**: آرایه ای از هزینه ها در طول زمان که برای رسم نمودار همگرایی استفاده میشود.
- **states_history**: تاریخچه ی موقعیت ها برای خطایابی و تصویرسازی مپ.

```
evaluations.append(curr_cost)
states_history.append(copy.deepcopy(curr_state))

# سیستم رو سرد میکنیم
t *= cooling_rate

return best_state, best_cost, evaluations, states_history
```

نقاط قوت و ضعف الگوریتم:

نقاط قوت:

- دقت بسیار بالا در نقشه‌های پیچیده: سنسورها می‌توانند با پرش از موانع محلی و فواصل، چیدمان‌های بهینه‌ای را در مپ‌های بزرگ پیدا کنند که Hill Climbing هرگز به آن‌ها دسترسی نداشت.
- عدم وابستگی شدید به حالت اولیه: حتی اگر چیدمان اولیه تصادفی بسیار ضعیف باشد، دمای بالای اولیه اثر آن را خنثی کرده و برخلاف الگوریتم Hill climbing نقشه را مجدد کاوش می‌کند تا به جواب بهینه سراسری برسد.

نقاط ضعف:

- تنظیم سخت مقداردهی پارامترها : کارایی الگوریتم به شدت به انتخاب دقیق initial_temp و cooling_rate وابسته است. اگر سیستم سریع سرد شود (مثلاً نرخ ۰.۸)، رفتار آن شبیه Hill Climbing شده و در بهینه محلی متوقف می‌شود. اگر دیر سرد شود، زمان اجرای برنامه بسیار طولانی می‌شود.

الگوریتم ژنتیک (Genetic Algorithm):

کارکرد الگوریتم:

الگوریتم ژنتیک (GA) یک روش بهینه‌سازی و جستجوی سراسری است که از اصول تکامل طبیعی ژنتیک الگوبرداری شده است. برخلاف الگوریتم‌های Hill climbing و Simulated Annealing که در هر لحظه تنها یک حالت (Single State) را بهبود می‌دهند، الگوریتم ژنتیک روی مجموعه‌ای از پاسخ‌ها (Population) کار می‌کند. این ویژگی قابلیت کاوش همزمان در بخش‌های مختلف نقشه را به ما می‌دهد.

حال با این ویژگی‌های اصول تکامل می‌توانیم متغیرهایی را تعریف کنیم و با اصول تکامل ژنتیک متغیر را تبدیل کنیم:

- کروموزوم / فرد (Chromosome/Individual): یک لیست از مختصات سنسورها (state).
- جمعیت (Population): مجموعه‌ای از چیدمان‌های مختلف سنسورها.

- تابع برازندگی (Fitness Function): معکوس یا منفی تابع هزینه (در اینجا از همان تابع هزینه استفاده شده و هدف کمینه‌سازی آن است).

توضیح و تحلیل کد الگوریتم:

همان طور که در الگوریتم‌های قبلی توضیح دادیم کلاس ژنتیک نیز متدهای کلاس LocalSearchBase را به ارث می‌برد. متغیرها را تعریف می‌کنیم. الگوریتم برای حفظ پیوستگی با نقطه شروع، حالت اولیه پیشنهادی (initial_state) را به عنوان عضو اول وارد جمعیت می‌کند و مابقی جمعیت را تا سقف population_size (به طور پیش‌فرض ۴۰ فرد) با چیدمان‌های تصادفی و معتبر پر می‌کند.

```
class GeneticAlgorithm(LocalSearchBase):
    def run(self, initial_state, population_size=40, generations=200,
            mutation_rate=0.3, tournament_size=4, **kwargs):

        # پر کردن جمعیت اولیه با استیت فعلی و بقیه به صورت رندوم
        pop = [copy.deepcopy(initial_state)]
        while len(pop) < population_size:
            pop.append(self.initialize_state())

        best_state = copy.deepcopy(initial_state)
        best_cost = self.evaluate(best_state)

        evaluations = []
        states_history = []
```

سپس الگوریتم با محاسبه کردن میزان هزینه هر عضو و مرتب کردن آنها می‌تواند بهترین عضوها را پیدا کند و در نتیجه بهترین جواب را در آن نسل پیدا کند.

```
for _ in range(generations):
    # محاسبه کاست هر عضو و سورت کردن از کم به زیاد
    pop_costs = [(state, self.evaluate(state)) for state in pop]
    pop_costs.sort(key=lambda x: x[1])

    # آپدیت کردن بهترین جواب کلی در صورت پیدا شدن حالت بهتر
    if pop_costs[0][1] < best_cost:
        best_cost = pop_costs[0][1]
        best_state = copy.deepcopy(pop_costs[0][0])

    evaluations.append(best_cost)
    states_history.append(copy.deepcopy(best_state))
```

پس از مرتب‌سازی جمعیت بر اساس هزینه، دو چیدمان برتر (کم‌هزینه‌ترین‌ها) مستقیم و بدون تغییر به نسل بعدی منتقل می‌شوند. این کار تضمین می‌کند که سیستم بهترین پاسخ‌های کشف‌شده تا به این لحظه را بر اثر ترکیب یا

جهش‌های تصادفی ناخواسته در نسل‌های بعدی از دست نخواهد داد. برای تولید بقیه جمعیت، از روش انتخاب تورنمنت (tournament_selection) با اندازه k (در اینجا 4) استفاده می‌کنیم که در ادامه توضیح داده خواهد شد. سپس از تابع crossover برای تولید فرزند جدید استفاده می‌کنیم.

```
# انتقال دو تا از بهترین‌ها به نسل بعد (Elitism)
new_pop = [
    copy.deepcopy(pop_costs[0][0]),
    copy.deepcopy(pop_costs[1][0])
]

# تولید بقیه جمعیت نسل جدید با کراس‌اور و میوتیشن
while len(new_pop) < population_size:
    p1 = self._tournament_selection(pop_costs, tournament_size)
    p2 = self._tournament_selection(pop_costs, tournament_size)

    child = self._crossover(p1, p2)

    if random.random() < mutation_rate:
        child = self._mutate(child)

    new_pop.append(child)

pop = new_pop

return best_state, best_cost, evaluations, states_history
```

تابع تورنمنت را برای انتخاب والدین جهت تولید نسل جدید تعریف کرده و اندازه آن را 4 در نظر می‌گیریم. روش کار تابع به این صورت است که 4 نفر را به صورت تصادفی از جمعیت انتخاب کرده و آن‌ها را با هم مقایسه می‌کند؛ فردی که کمترین هزینه را داشته باشد انتخاب شده و به عنوان والد بازگردانده می‌شود.

```
def _tournament_selection(self, pop_costs, k):
    # عضو تصادفی و برگزیدن اونی که کاستش کمتره k انتخاب
    tournament = random.sample(pop_costs, k)
    tournament.sort(key=lambda x: x[1])
    return copy.deepcopy(tournament[0][0])
```

تابع crossover با ترکیب سنسورهای دو والد و حذف نقاط تکراری، فرزند جدید را می‌سازد و محدودیت تعداد سنسورها را اعمال می‌کند.

```
def _crossover(self, p1, p2):
    # ترکیب نصف سنسورهای والد اول و نصف والد دوم
    sp1 = len(p1) // 2
    sp2 = len(p2) // 2

    child = p1[:sp1] + p2[sp2:]
    child = list(set(child)) # حذف سنسورهای تکراری در یک نقطه

    # رعایت محدودیت حداکثر تعداد سنسورها
    if len(child) > self.max_sensors:
        child = random.sample(child, self.max_sensors)

    if len(child) == 0:
        child = self.initialize_state()

    return child
```

تابع جهش (mutate) برای ایجاد تنوع و فرار از بهینه‌های محلی استفاده می‌شود. در ابتدای کد نرخ جهش (mutation_rate) را روی 0.3 تنظیم کرده بودیم یعنی هر فرزند ۳۰٪ شانس دارد که دچار جهش شود.

تابع جهش از متد get_neighbor کلاس LocalSearchBase استفاده می‌کند. به این معنا که جهش فرزند می‌تواند به صورت تصادفی باعث حرکت دادن یک سنسور، اضافه کردن سنسور جدید یا حذف یکی از سنسورها شود.

```
def _mutate(self, state):
    # اعمال جهش روی فرزند با استفاده از متد همسایگی
    return self.get_neighbor(state)
```

نقاط قوت و ضعف الگوریتم:

نقاط قوت:

- موازی بودن جستجو: به دلیل بررسی همزمان ۴۰ چیدمان مختلف در مپ، احتمال پیدا کردن فضاهای کور نقشه و نقاط کلیدی بالاتر از روش‌های تک‌حالتی است.
- انعطاف داشتن در ساختار پاسخ: به دلیل طراحی داینامیک crossover، الگوریتم می‌تواند ساختارهایی با تعداد سنسورهای متفاوت را با هم ترکیب کند و بهترین تعداد و موقعیت را از آن استخراج کند.

نقاط ضعف:

- بار محاسباتی سنگین: در هر نسل (Generation) باید به تعداد حجم جمعیت، تابع هزینه فراخوانی شود. این موضوع بار محاسباتی زیادی را تحمیل سیستم می‌کند.

الگوریتم Beam Search:

کارکرد الگوریتم:

الگوریتم Beam Search نسخه‌ای بهینه‌شده و موازی از الگوریتم Hill climbing است. در Hill climbing، اگر الگوریتم مسیر را اشتباه برود در بهینه محلی متوقف می‌شود و نمی‌تواند از آن حالت خارج شود. اما Beam search با تعریف مفهومی به نام Beam Width که با k نشان داده می‌شود، در هر گام به جای یک حالت، k حالت برتر را به طور همزمان پیش می‌برد. در جستجوی موازی ساده، چندین رشته‌ی Hill climbing مستقل از هم کار می‌کنند و هیچ تبادلی ندارند. اما در Beam search، اطلاعات در هر گام sync می‌شوند. یعنی همه‌ی همسایه‌ها با هم مقایسه شده و فقط بهترین‌های کل مجموعه مجاز به ادامه مسیر هستند. در واقع این کار نوعی اشتراک اطلاعات محسوب می‌شود.

توضیح و تحلیل کد الگوریتم:

الگوریتم ابتدا حالت پیشنهادی ورودی را در پرتو (beam) قرار می‌دهد و بقیه ظرفیت پرتو را تا سقف beam_width (به طور پیش‌فرض ۵ چیدمان) با استفاده از تابع initialize_state کلاس LocalSearchBase، با چیدمان‌های تصادفی معتبر پر می‌کند. سپس بهترین آن‌ها را به عنوان نقطه شروع ثبت می‌کند.

```
class BeamSearch(LocalSearchBase):
    def run(self, initial_state, beam_width=5, max_iterations=100, **kwargs):
        # پر کردن پرتو (بیم) با استیت فعلی و بقیه به صورت رندوم
        curr_states = [copy.deepcopy(initial_state)]
        while len(curr_states) < beam_width:
            curr_states.append(self.initialize_state())

        best_state = min(curr_states, key=self.evaluate)
        best_cost = self.evaluate(best_state)

        evaluations = [best_cost]
        states_history = [copy.deepcopy(best_state)]
```

در هر تکرار از حلقه اصلی (تا سقف $\text{max_iterations}=100$)، الگوریتم برای هر کدام از حالت‌های موجود در پرتو، ۱۲ همسایه تولید می‌کند. با توجه به پهنای پرتو پیش‌فرض (۵) که در نظر گرفته شده، در هر گام مجموعاً $5 \times 12 = 60$ همسایه جدید در لیست `all_nbs` جمع‌آوری می‌شود.

```
# تولید ۱۲ همسایه برای هر کدام از وضعیت‌های داخل بیم
for state in curr_states:
    for _ in range(12):
        all_nbs.append(self.get_neighbor(state))
```

پس از جمع‌آوری تمام ۶۰ همسایه، کل این لیست بر اساس تابع هزینه (از کم به زیاد) را مرتب می‌کند. سپس، الگوریتم بدون توجه به اینکه هر همسایه متعلق به کدام والد بوده است، ۵ همسایه برتر کل نقشه را به عنوان پرتو جدید (`curr_states`) برای گام بعدی انتخاب می‌کند.

از آنجا که لیست همسایه‌ها مرتب شده است، عنصر اول این لیست یعنی `curr_states[0]` بهترین حالت یافت شده در این گام است. اگر هزینه‌ی این حالت از بهترین هزینه‌ی کل (`best_cost`) کمتر باشد، به عنوان پاسخ برتر سیستم ذخیره می‌شود.

```
# سورت کردن کل همسایه‌ها بر اساس کاست
all_nbs.sort(key=self.evaluate)

# برای نسل بعد (beam_width) انتخاب بهترین‌ها به تعداد پهنای بیم
curr_states = copy.deepcopy(all_nbs[:beam_width])

step_best_cost = self.evaluate(curr_states[0])

# آپدیت کردن بهترین جواب کلی
if step_best_cost < best_cost:
    best_cost = step_best_cost
    best_state = copy.deepcopy(curr_states[0])

evaluations.append(best_cost)
states_history.append(copy.deepcopy(best_state))

return best_state, best_cost, evaluations, states_history
```

نقاط قوت و ضعف الگوریتم:

نقاط قوت:

○ تعادل بین سرعت و کیفیت: این الگوریتم به دلیل تمرکز حریصانه روی بهترین همسایه‌ها، سرعت همگرایی بسیار بالایی دارد و در عین حال به خاطر داشتن ۵ پرتو همزمان، شانس گیر افتادن آن در بهینه‌های محلی ضعیف، بسیار کمتر از Hill climbing معمولی است.

○ مدیریت موازی موانع مپ: اگر یکی از پرتوها در یک بخش بن‌بست یا کم‌بازده از نقشه گیر بیفتد، در گام بعدی مرتب‌سازی، همسایه‌های آن پرتو خودبه‌خود حذف می‌شوند و پرتوهای موفق‌تر کنترل جستجو را به دست می‌گیرند.

نقاط ضعف:

○ کاهش تنوع: اگر یکی از حالت‌ها به شدت قوی‌تر از بقیه باشد، ممکن است در گام سورتینگ، تمام ۵ همسایه‌ی برتر انتخاب‌شده، فرزندان همان یک حالت باشند. در این صورت، پرتوها همگرا شده و عملاً الگوریتم به یک Hill climbing ساده تبدیل می‌شود.

الگوریتم Tabu Search:

کارکرد الگوریتم:

الگوریتم Tabu Search (یا به اختصار TS) بر اساس مفهوم حافظه (Memory) کار می‌کند. بزرگترین ضعف Hill climbing این است که وقتی به یک بهینه محلی رسید، متوقف می‌شود و حتی اگر حرکات هم‌سطح (Flat Plateau) را بپذیرد، ممکن است دچار حلقه‌ی بی‌نهایت شود و مدام بین چند وضعیت تکراری جابه‌جا شود.

Tabu search با ثبت آخرین وضعیت‌های دیده شده در یک لیست حافظه کوتاه‌مدت به نام لیست ممنوعه (Tabu List)، بازگشت به آن وضعیت‌ها را برای مدتی محدود (به اندازه tabu_size) ممنوع می‌کند. این مکانیزم الگوریتم را مجبور می‌کند تا از چاله‌های بهینه محلی خارج شده و نواحی جدیدی از نقشه را کاوش کند.

توضیح و تحلیل کد الگوریتم:

ابتدا، حالت اولیه کپی شده و هزینه آن محاسبه می‌شود. دو متغیر `best_state` و `best_cost` را تعریف می‌کنیم تا بهترین چیدمانی که سیستم در کل طول اجرای خود (نه فقط گام فعلی) پیدا کرده است را در خود نگه دارند.

از آنجایی که در پایتون وضعیت‌ها (`states`) به صورت لیست ذخیره می‌شوند و لیست‌ها به دلیل تغییرپذیر بودن (`Mutable`)، قابل هش شدن نیستند، نمی‌توان تکراری بودن آن‌ها را در یک مجموعه یا لیست دیگر به سرعت بررسی کرد. پس برای بررسی تکراری بودن یا نبودن یک چیدمان در این کد، تابع بی‌نام (`lambda`) را تعریف می‌کنیم. این تابع هر حالت را ابتدا `sort` کرده و سپس به تاپل تبدیل می‌کند. با این کار، تاپل‌ها غیرقابل تغییر (`Immutable`) و در نتیجه قابل هش شدن هستند. علاوه بر آن، چیدمان سنسورها مستقل از ترتیب ایندکس‌هایشان سنجیده می‌شود؛ یعنی دو حالت `[(1,2), (3,4)]` و `[(3,4), (1,2)]` که از نظر فیزیکی روی نقشه یکسان هستند، کلید یکسانی تولید می‌کنند. در نهایت، کلید حالت اولیه وارد `tabu_list` می‌شود.

```
class TabuSearch(LocalSearchBase):
    def run(self, initial_state, tabu_size=15, max_iterations=120, num_neighbors=25, **kwargs):

        curr_state = copy.deepcopy(initial_state)
        curr_cost = self.evaluate(curr_state)

        best_state = copy.deepcopy(curr_state)
        best_cost = curr_cost

        evaluations = [curr_cost]
        states_history = [copy.deepcopy(best_state)]

        # تبدیل استیت به تیوپل سورت‌شده واسه اینکه بتونیم تو لیست تابو چک کنیم
        to_key = lambda s: tuple(sorted(s))
        tabu_list = [to_key(curr_state)]
```

بعد از آن الگوریتم وارد حلقه اصلی (تا سقف ۱۲۰ تکرار) می‌شود. در هر گام، به تعداد `num_neighbors` (پیش‌فرض ۲۵ بار) متد `get_neighbor` از کلاس `LocalSearchBase` صدا زده می‌شود تا تغییراتی مثل حرکت، حذف یا اضافه کردن سنسور را روی حالت فعلی اعمال کند.

اطلاعات هر همسایه شامل خودِ حالت، هزینه آن و کلید منحصر به فردش در لیست `candidates` ذخیره شده و در پایان با دستور `sort` بر اساس هزینه از کم به زیاد مرتب می‌شوند تا بهترین همسایه‌ها در ابتدای لیست قرار بگیرند.

```

تبدیل استیت به تیوپل سورت شده واسه اینکه بتونیم تو لیست تابو چک کنیم
to_key = lambda s: tuple(sorted(s))
tabu_list = [to_key(curr_state)]

for _ in range(max_iterations):
    candidates = []

    تولید همسایه‌ها و حساب کردن کاست و کلید هر کدام
    for _ in range(num_neighbors):
        nb = self.get_neighbor(curr_state)
        nb_cost = self.evaluate(nb)
        nb_key = to_key(nb)
        candidates.append((nb, nb_cost, nb_key))

    candidates.sort(key=lambda x: x[1])

```

الگوریتم کاندیداها را به ترتیب از بهترین به بدترین بررسی می‌کند. شرط انتخاب یک همسایه این است که کلید آن در لیست ممنوعه نباشد ($nb_key \text{ not in } tabu_list$). بخش دوم شرط یعنی $best_cost > nb_cost$ یعنی اگر یک همسایه در لیست ممنوعه باشد، اما هزینه‌ی آن از بهترین هزینه‌ای که کل سیستم تا به حال به چشم دیده کمتر باشد، قانون ممنوعیت لغو می‌شود و الگوریتم آن را می‌پذیرد. چراکه منطقی نیست پاسخی که رکورد کل را بهبود می‌دهد، فقط به دلیل ملاقات اخیر رد شود.

```

chosen = None
for nb, nb_cost, nb_key in candidates:
    if nb_key not in tabu_list or nb_cost < best_cost:
        chosen = (nb, nb_cost, nb_key)
        break

```

اگر در یک شرایط خاص، تمام ۲۵ کاندیدای تولید شده همگی جزو لیست ممنوعه باشند و هیچ کدام هم شرط ابطال را برآورده نکنند، متغیر **chosen** مقدار **None** باقی می‌ماند. در این حالت، برای جلوگیری از قفل شدن برنامه، الگوریتم به ناچار اولین (بهترین) کاندیدای موجود را انتخاب می‌کند تا حرکت در نقشه متوقف نشود.

```

# که همه کاندیداها تابو بودن، مجبوری اولین (بهترین) رو برمی‌داریم
if chosen is None and candidates:
    chosen = candidates[0]

```

حالت فعلی سیستم به کاندیدای برگزیده جابه‌جا می‌شود (حتی اگر هزینه‌ی آن از حالت قبلی بدتر باشد). اگر این حالت جدید رکورد کل را جابه‌جا کند، به عنوان بهترین پاسخ سراسری ذخیره می‌شود.

```

if chosen:
    curr_state, curr_cost, chosen_key = chosen

    # آپدیت بهترین جواب کلی
    if curr_cost < best_cost:
        best_cost = curr_cost
        best_state = copy.deepcopy(curr_state)

```

کلید حالت جدید به انتهای لیست ممنوعه اضافه می‌شود تا در گام‌های بعدی مجدداً تکرار نشود. برای اینکه سیستم قفل نشود، اندازه این حافظه کوتاه‌مدت با `tabu_size=15` کنترل می‌شود. اگر طول لیست از ۱۵ بیشتر شود، قدیمی‌ترین عضو با متد `pop(0)` حذف می‌شود (مکانیزم First-In-First-Out) تا پس از ۱۵ گام، بازگشت به آن نقطه مجاز شود.

```

# اضافه کردن به لیست تابو و رعایت سقف اندازه لیست
tabu_list.append(chosen_key)
if len(tabu_list) > tabu_size:
    tabu_list.pop(0)

evaluations.append(best_cost)
states_history.append(copy.deepcopy(best_state))

return best_state, best_cost, evaluations, states_history

```

نقاط قوت و ضعف الگوریتم:

• نقاط قوت:

- جلوگیری از درجا زدن و حلقه (Cycling Prevention): به دلیل ماهیت حافظه‌محور، سنسورها هرگز بین دو موقعیت یکسان مدام رفت‌ووبرگشت نمی‌کنند و مجبور به پیشروی در مپ می‌شوند.
- بهره‌برداری از فلات‌ها (Plateaus): اگر الگوریتم به بخشی از نقشه برسد که تغییر موقعیت سنسورها هزینه‌ی یکسانی دارد، لیست ممنوعه سنسورها را وادار می‌کند تا به جلو حرکت کنند تا بالاخره راه بهتری پیدا شود.

• نقاط ضعف:

- مصرف حافظه و زمان پردازش کلیدها: تبدیل وضعیت‌ها به تاپل‌های مرتب‌شده و جستجو در لیست ممنوعه در نقشه‌های بسیار بزرگ با تعداد بالای سنسورها، کار پردازشی سیستم را افزایش می‌دهد.

در نهایت، در کلاس `main` کلاس‌های الگوریتم‌ها را `import` می‌کنیم:

```
from env.grid_world import GridWorld
from search.hill_climbing import HillClimbing
from search.simulated_annealing import SimulatedAnnealing
from search.genetic_algorithm import GeneticAlgorithm
from search.beam_search import BeamSearch
from search.tabu_search import TabuSearch
from utils import represent
```

در این قسمت هم مشخص می‌کنیم که نقشه (`map`) ما کدام است و کلاس‌ها را ران می‌کنیم:

```
if __name__ == "__main__":
    world = GridWorld("map3")

    algorithm_classes = [
        HillClimbing,
        SimulatedAnnealing,
        GeneticAlgorithm,
        BeamSearch,
        TabuSearch
    ]
```

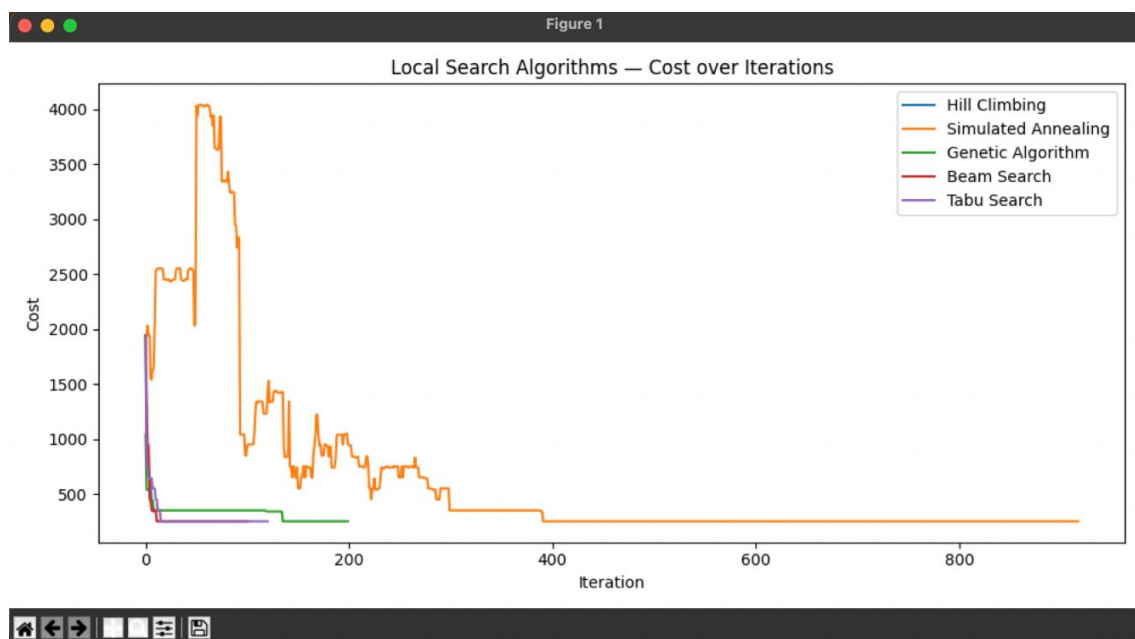
جمع‌بندی الگوریتم‌ها:

- **Hill Climbing:** کاملاً حریصانه و بدون آینده‌نگری عمل می‌کند. سنسورها را فقط به سمت بهبود لحظه‌ای پوشش حرکت می‌دهد و به محض رسیدن به اولین بهینه محلی متوقف می‌شود. سرعت بالا اما دقت پایینی در نقشه‌های پیچیده دارد.
- **Simulated Annealing:** در ابتدا (در دمای بالا) آگاهانه پاسخ‌های بدتر و چیدمان‌های ضعیف را می‌پذیرد تا سنسورها بتوانند از موانع و بهینه محلی عبور کنند. با سرد شدن تدریجی، رفتار سیستم حریصانه می‌شود. این روش تعادلی عالی بین زمان اجرا و فرار از تله (بهینه محلی) است.

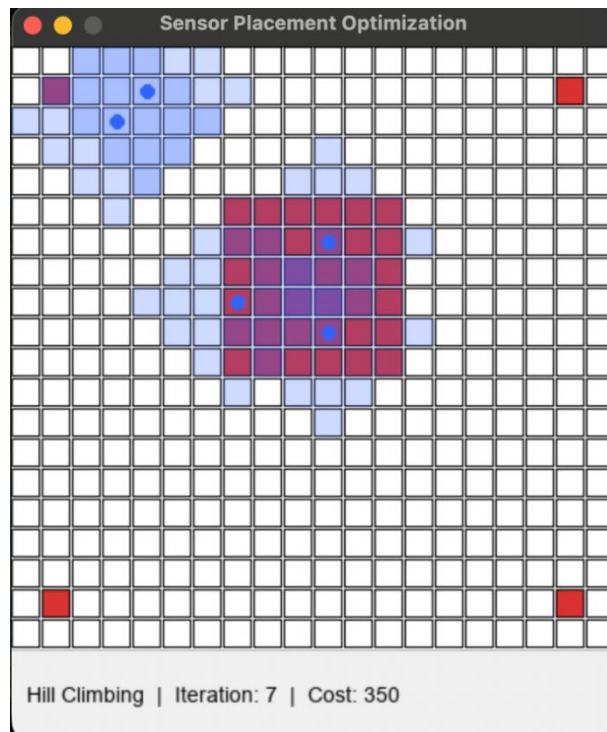
- الگوریتم ژنتیک (Genetic Algorithm): به جای یک چیدمان، روی یک جمعیت ۴۰ تایی کار می‌کند. با ترکیب سنسورهای بهینه در نقاط مختلف نقشه (crossover) و ایجاد جهش‌های تصادفی، بالاترین شانس را برای کشف بهترین چیدمان سراسری دارد، اما از طرفی بیشترین زمان را برای پردازش استفاده می‌کند.
- Beam Search: چیدمان برتر را به صورت موازی جلو می‌برد. در هر گام ۶۰ همسایه را ارزیابی کرده و بهترین‌های کل نقشه را انتخاب می‌کند. اگر سنسورهای یک پرتو در نقطه کوری گیر بیفتند، آن پرتو حذف شده و پرتوهای موفق‌تر جایگزین می‌شوند.
- Tabu Search: با استفاده از یک حافظه ۱۵ تایی، بازگشت به چیدمان‌های اخیر را ممنوع می‌کند. این مکانیزم سنسورها را وادار می‌کند تا از درجا زدن و چرخش در مپ دست بردارند و به سمت نواحی ناشناخته پیش بروند.

خروجی کدها:

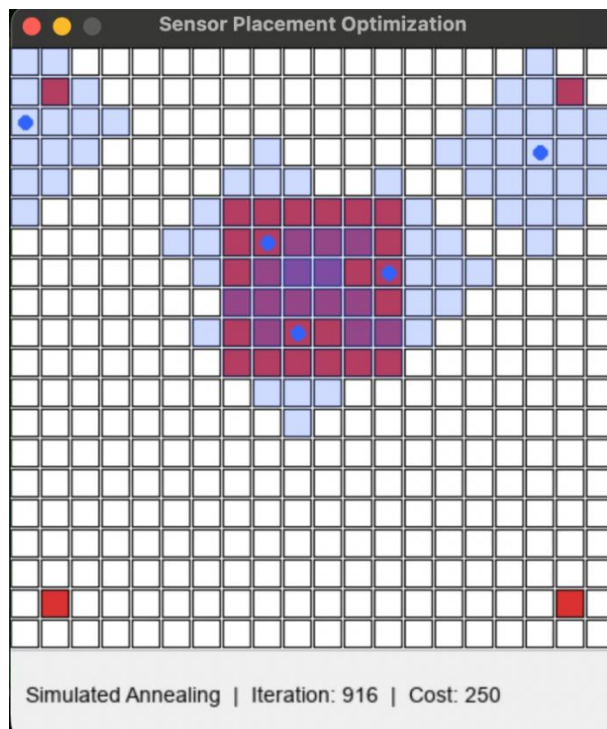
الگوریتم‌ها برای ۷ نقشه پیش فرض به درستی کار می‌کند. تصویر خروجی‌های زیر طبق نقشه شماره ۳ هستند:



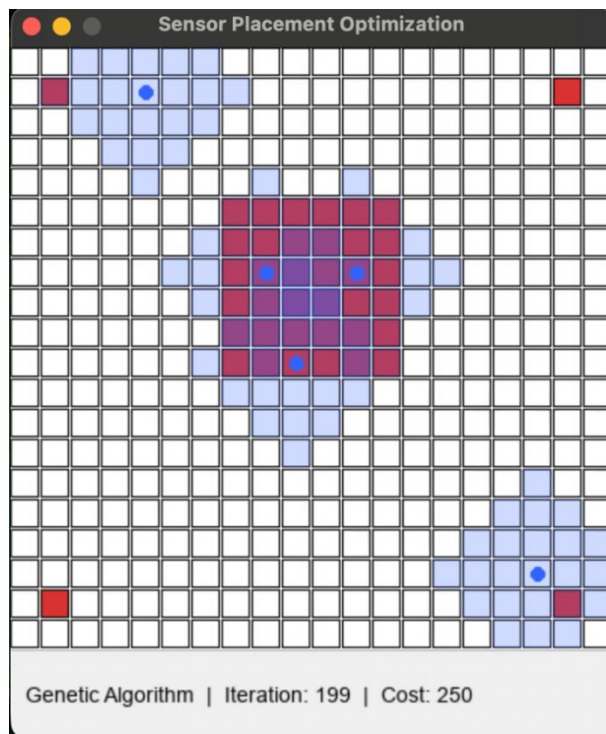
نمودار همگرایی



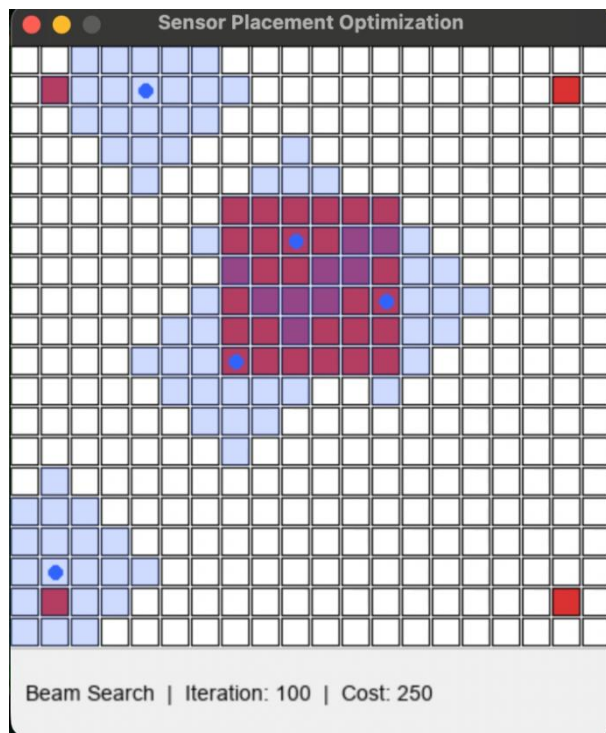
تصویر جایگذاری سنسور ها با الگوریتم Hill Climbing



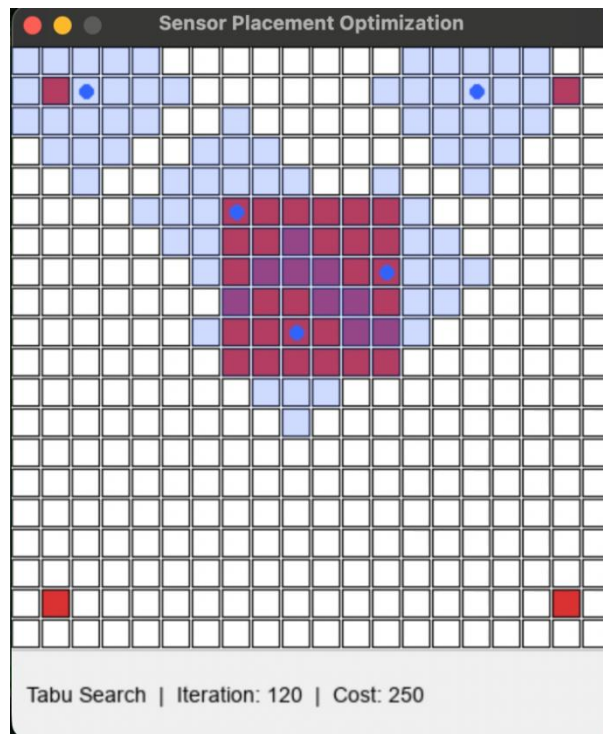
تصویر جایگذاری سنسور ها با الگوریتم Simulated Annealing



تصویر جایگذاری سنسور ها با الگوریتم Genetic



تصویر جایگذاری سنسور ها با الگوریتم Beam Search



تصویر جایگذاری سنسورها با الگوریتم Tabu Search

در ترمینال خروجی‌های زیر را خواهیم داشت:

```
➔ 16c083_9bf7 /usr/local/bin/python3 /Users/farnoush/Downloads/16c083_9bf7/main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.4)
Hello from the pygame community. https://www.pygame.org/contribute.html
Running Hill Climbing...
Running Simulated Annealing...
Running Genetic Algorithm...
Running Beam Search...
Running Tabu Search...
```

Hill Climbing

```
best_state : [(2, 3), (6, 10), (9, 10), (8, 7)]
best_cost : 340
evaluations : [1940, 950, 750, 740, 650]...
(Total Steps: 9 | Initial Cost: 1940 | Final Cost: 340)
states_history : 9 states recorded | Best state sensors: 4
```

Simulated Annealing

```
best_state : [(8, 12), (0, 19), (5, 8), (10, 7), (0, 3)]
best_cost : 250
evaluations : [1940, 1940, 2030, 1940, 1940]...
(Total Steps: 918 | Initial Cost: 1940 | Final Cost: 250)
states_history : 918 states recorded | Best state sensors: 5
```

Genetic Algorithm

```
best_state : [(17, 17), (1, 4), (10, 9), (7, 11), (7, 8)]
best_cost : 250
evaluations : [1040, 540, 540, 530, 530]...
(Total Steps: 200 | Initial Cost: 1040 | Final Cost: 250)
states_history : 200 states recorded | Best state sensors: 5
```

Beam Search

```
best_state : [(6, 9), (1, 4), (10, 7), (8, 12), (17, 1)]
best_cost : 250
evaluations : [1940, 1350, 950, 940, 450]...
(Total Steps: 101 | Initial Cost: 1940 | Final Cost: 250)
states_history : 101 states recorded | Best state sensors: 5
```

Tabu Search

```
best_state : [(7, 12), (1, 2), (9, 9), (1, 15), (5, 7)]
best_cost : 250
evaluations : [1940, 750, 740, 650, 640]...
(Total Steps: 121 | Initial Cost: 1940 | Final Cost: 250)
states history : 121 states recorded | Best state sensors: 5
```